

Formatting Clear Java Program Output

Generated Jul 30, 2026 6:35 AM

What We'll Learn

By the end of this lesson, students can:

- Use `print()`, `println()`, and `printf()` to display output.
- Add labels, spaces, and line breaks to make output easier to read.
- Combine text and variable values in one output statement.
- Format decimal numbers to a specific number of decimal places.
- Organize program results clearly.

Before We Start

Imagine that a program displays the following result:

```
Juan1789.5
```

What information does this output show?

How could we organize it so that the user can understand it more easily?

Let's Understand

A program's output should be clear, organized, and easy to understand. Java provides several ways to display output.

Using `System.out.print()`

The `print()` method displays output without moving to a new line.

```
System.out.print("Hello, ");  
System.out.print("Juan!");
```

Output:

```
Hello, Juan!
```

Using `System.out.println()`

The `println()` method displays output and moves the next output to a new line.

```
System.out.println("Student Information");  
System.out.println("Name: Juan");  
System.out.println("Grade Level: 11");
```

Output:

```
Student Information  
Name: Juan  
Grade Level: 11
```

Combining Text and Variables

The `+` operator can combine text and variable values.

```
String name = "Juan";  
int age = 17;  
  
System.out.println("Name: " + name);  
System.out.println("Age: " + age);
```

Labels such as `Name:` and `Age:` help the user understand what each value represents.

Using Escape Sequences

Escape sequences are special characters that control how output appears.

- `\n` – starts a new line
- `\t` – adds a tab space

- `\"` – displays quotation marks
- `\\` – displays a backslash

Example:

```
System.out.println("Name:\tJuan");  
System.out.println("Section:\tProgramming 11");
```

Output:

```
Name:      Juan  
Section:   Programming 11
```

Formatting Decimal Numbers

The `printf()` method can control how numbers are displayed.

```
double average = 89.4567;  
  
System.out.printf("Average: %.2f%n", average);
```

Output:

```
Average: 89.46
```

The format specifier `%.2f` displays a decimal number with two digits after the decimal point.

Common format specifiers include:

- `%s` – text or `String`
- `%d` – whole number or `int`
- `%f` – decimal number
- `%.2f` – decimal number with two decimal places
- `%n` – new line

Example:

```
String name = "Maria";  
int score = 95;  
  
System.out.printf("Name: %s%n", name);  
System.out.printf("Score: %d%n", score);
```

Clear output should:

- Include meaningful labels.
- Use proper spacing and line breaks.
- Display related information together.

- Format numbers appropriately.
- Avoid unnecessary or confusing text.

Let's Practice

Create a Java program that stores the following information in variables:

- Customer name
- Product name
- Quantity
- Price per item

Calculate the total price and display the information using this format:

```
===== SALES RECEIPT =====  
Customer: Maria Cruz  
Product: Ballpen  
Quantity: 5  
Price: PHP 12.50  
-----  
Total: PHP 62.50  
=====
```

Your program must:

- Use meaningful variable names.
- Calculate the total price.
- Use labels and line breaks.
- Use `printf()` to display prices with two decimal places.
- Organize the output so that it is easy to read.

Let's Reflect

Why are labels, spacing, line breaks, and decimal formatting important when displaying the results of a Java program?